IoTShield

MINI PROJECT – CRYPTOGRAPHY

SAII (साई) SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE

Project Title: IoTShield – Secure IoT Device Authentication using HMAC-SHA256 Challenge–Response, with Signed Firmware and Shared Key Custody

Student Name(s): Gargi Deore, Diya Jairamani | **Roll No(s):** 25030421013, 25030421011 | **Class/Division:** B.Sc | **Faculty Guide:** Dr. Reema Roychoudhary | **Department:** SAII | **Academic Year:** 2026 – 27

1. PROBLEM STATEMENT

The problem. IoT devices (sensors, cameras, GPS trackers, smart locks) talk to servers over networks anyone can listen to. A server that only trusts a device ID, or a password sent over the network, can be fooled by **spoofing**, by a **replayed** recording of a valid login, or by one **leaked shared secret** that exposes every device. Attackers also push **malicious firmware** that devices install blindly, and the one **master key** that protects every device secret is a single file that can be stolen or lost.

Why it matters. Unattended, always-on devices are easy doors into homes, factories and vehicle fleets. One forged identity or poisoned update can compromise the whole network.

The need. Prove a device owns a secret **without ever sending the secret**, prove an update really comes from its maker, and make sure **no single person or file** holds the master key.

2. OBJECTIVES

1. Authenticate every device by **proof of key ownership**: the secret key is never transmitted.
2. Make recorded or forged responses useless with **single-use random challenges** that expire in 2 minutes.
3. **Protect the keys:** device keys encrypted at rest (Fernet); the master key can be **split among people** (Shamir), sealed off the disk and rotated; admin login needs a password + emailed code.
4. Accept firmware only with a valid **Ed25519 signature**, and record every attempt in an **audit log**.

3. CRYPTOGRAPHIC CONCEPT

Algorithm / Protocol:
Challenge–response with **HMAC-SHA256** (RFC 2104) · **Ed25519** signatures (RFC 8032) · **Fernet** authenticated encryption · **Shamir's Secret Sharing** (k-of-n)

Type:
Symmetric MAC (device login) · asymmetric digital signature (firmware) · symmetric AEAD, AES-128-CBC + HMAC (keys at rest) · secret sharing for the master key

Key / Parameters:
256-bit random device key · 256-bit random challenge, one use, 120 s expiry · 32-byte Ed25519 key pair · master key split 3-of-5 over a 521-bit prime field · 6-digit admin code

Security Properties:
Authentication · integrity · freshness (anti-replay) · constant-time check · confidentiality of stored keys · firmware origin · **k-1 shares reveal nothing**

4. REAL-WORLD APPLICATIONS

- **Secure Communication:** prove identity before a device may talk to a server
- **Data Protection:** keys encrypted at rest; master key split among custodians, as in key-management systems
- **Authentication:** smart locks, sensors, cameras, GPS trackers
- **Digital Signatures:** signed firmware / over-the-air updates
- **E-commerce / Banking:** HMAC-signed API calls, emailed one-time codes (2FA)
- **IoT / Cloud Security:** authenticate and audit a whole fleet
- **Others:** smart homes, industrial IoT, vehicle fleets

5. SYSTEM ARCHITECTURE / DESIGN

A · DEVICE LOGIN (CHALLENGE–RESPONSE)

IoT Device (Input) "I am TEMP-001" → Random Challenge 32 bytes, from secrets, valid once, 2 min → Secret Key 256-bit, kept in the device, never sent → HMAC-SHA256 (Algorithm) key + challenge → Server Verifies recompute + constant-time compare → GRANTED / DENIED (Output) logged to SQLite

B · SIGNED FIRMWARE UPDATE (PUBLIC-KEY SIGNATURE)

Admin Uploads firmware image → SHA-256 hash of the image → Ed25519 Sign private key stays on the server → Device Downloads image + signature → Verify + Anti-rollback built-in public key → Install / Refuse HMAC-signed report

C · KEY CUSTODY (SHAMIR SECRET SHARING)

Master Key 256-bit, encrypts every device key → Split k-of-n 3 of 5 shares, shown once, never stored → Custodians each person keeps one share → Seal key file deleted from the disk → Unseal 3 shares rebuild it, in memory only → Rotate new key, all device keys re-encrypted

Admin browser (HTML · CSS · JS) ⇄ **Flask REST API** (app.py) ⇄ **SQLite** (devices · challenges · logs · firmware) · crypto.py · shamir.py · custody.py hold the cryptography and key custody

6. METHODOLOGY / WORKFLOW

STEP 1 — Requirement Analysis
threats: spoofing, replay, key theft, bad firmware

STEP 2 — Keys & Custody
256-bit keys, Fernet vault, Ed25519 pair, Shamir shares

STEP 3 — Challenge–Response
random challenge → HMAC → constant-time verify

STEP 4 — Signed Firmware
SHA-256 + Ed25519, version signed, no rollback

STEP 5 — Testing & Verification
88 automated tests, attack simulator

Attacks simulated, all refused and logged:

- Wrong key → **denied**
- Replayed response → **blocked**
- Forged challenge → **rejected**
- Bit-flipped firmware → **refused**
- Attacker-signed firmware → **refused**
- Old firmware relabelled as new → **refused**
- Downgrade to older version → **refused**
- Stolen key file → **vault sealed**
- Leaked share → **key rotated, old shares dead**

Every request, accepted or refused, is written to the security log so the administrator can audit it.

7. TECHNOLOGY STACK

Programming Language
Python 3 · JavaScript (ES6)
HTML5 · CSS3 · SQL

Tools / Platform
Flask 3 web framework · SQLite
REST / JSON API · browser dashboard
always-on background service (launchd)

Libraries
`hmac · hashlib · secrets (standard library)`
cryptography (Fernet, Ed25519) · python-dotenv
Shamir's scheme: our own implementation

Hardware (if any)
MacBook Air · simulated IoT devices
(sensor, camera, motion, GPS) via device_simulator.py

IoTShield

MINI PROJECT – CRYPTOGRAPHY

SAII(साई) SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE

8. IMPLEMENTATION

- **Approach:** Flask REST API + a device simulator that behaves like firmware over HTTP + a live dashboard
- **Modules:** crypto.py (all cryptography), database.py (schema), app.py (routes, protocol, rate limits)
- **Authentication:** challenge issued, HMAC recomputed on the server, compare_digest; 5 failures / 60 s lock the device
- **Firmware:** sign (version + SHA-256) with Ed25519; devices verify and refuse older versions
- **Key custody:** the master key is split into Shamir shares; **seal** deletes the key file, **unseal** rebuilds it in memory, **rotation** re-encrypts every device key in one crash-safe transaction
- **Admin console:** password + emailed 6-digit code, 5-minute idle sign-out, CSRF protection, live dashboard and audit log
- **Dataset & testing:** 4 demo devices with generated history; 88 automated tests plus an attack simulator

```python
# utils/crypto.py
def compute_hmac(secret_key_hex,
                 challenge_hex):
    key_bytes = bytes.fromhex(secret_key_hex)
    message = challenge_hex.encode("ascii")
    return hmac.new(key_bytes, message,
                    hashlib.sha256).hexdigest()

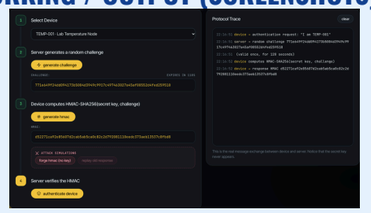
# server side, constant-time check
hmac.compare_digest(expected, received)

# device side, firmware check (Ed25519)
public_key.verify(
    signature,
    firmware_manifest(version, sha256_hex(image)))

# utils/shamir.py (secret at x = 0)
coefficients = [int.from_bytes(secret, "big")]
coefficients += [secrets.randbelow(PRIME)
                 for _ in range(threshold - 1)]
_evaluate(coefficients, x)   # one share per x

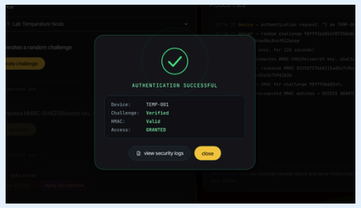
# utils/custody.py: a sealed vault
# never invents a new key
if meta.get("sealed"):
    ...
    self._vault = None   # sealed
```

9. WORKING / OUTPUT (SCREENSHOTS)

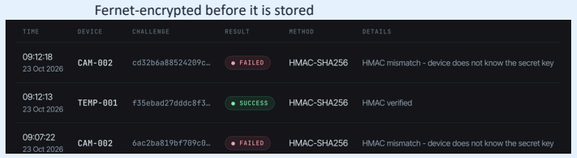


Screenshot 1 – Authentication. Challenge and HMAC are generated; the protocol trace shows the secret key never appears.

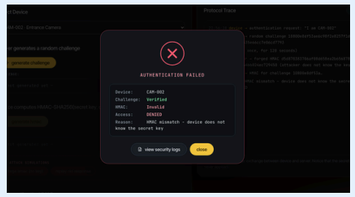
- **Key generation:** secrets.token_hex(32), then Fernet-encrypted before it is stored



Screenshot 2 – genuine device: HMAC valid, **access granted**



Screenshot 4 – Security log. Every attempt recorded: forged HMAC **FAILED**, replay blocked, genuine device **SUCCESS**.



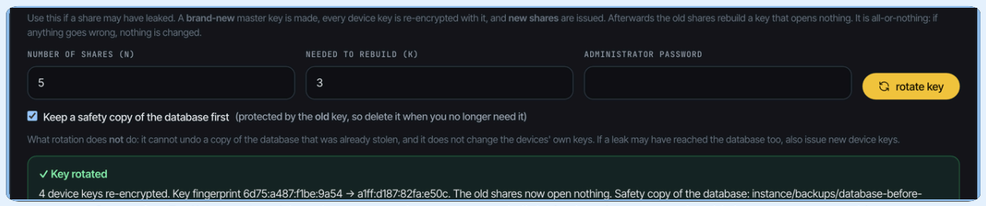
Screenshot 3 – forged HMAC (no key): **access denied**



Screenshot 5 – Signed firmware. Latest release, devices up to date, and **1 malicious update blocked**.



Screenshot 6 – Key custody. The master key was split into 5 shares: any 3 rebuild it, but with only 2 the test says **no match** and reveals nothing about the key.



Screenshot 7 – Key rotation. A brand-new master key re-encrypted all device keys in one step; the old shares now open nothing.

10. RESULTS & ANALYSIS

Test Case	Input Size	Result / Observation
1 Device key vault (Fernet)	64 B	Key recovered intact
2 HMAC-SHA256 login	32 B	ACCESS GRANTED
3 Forged HMAC (wrong key)	32 B	ACCESS DENIED
4 Signed firmware (Ed25519)	1 MB	Signature valid, installed
5 Firmware, 1 bit flipped	1 MB	Update refused
6 Replay of an old response	32 B	Blocked: challenge already used
7 Master key split (Shamir 3-of-5)	32 B	Any 3 shares rebuild it; 2 do not
8 Master-key rotation	1,000 keys	All re-encrypted; old shares refused

- The cryptography is not the bottleneck: a login's HMAC takes about **1 µs**, while the whole login (HTTP + database) takes **1.6 ms** .
- Checking a 1 MB firmware costs **0.45 ms**, mostly hashing the image
- Key custody costs almost nothing: splitting or rebuilding a key takes about **6 µs**, and rotating **1,000** device keys takes **26 ms** in one transaction.
- Key rotation survived simulated crashes **before and after** the database commit: on restart the server finished it by itself and every device key still opened.

11. SECURITY ANALYSIS

- **Confidentiality:** device keys are AES-encrypted (Fernet); the master key can be **sealed off the disk**, so a stolen disk holds only encrypted keys; keys never reach the browser or the network.
- **Integrity:** firmware signature covers version + SHA-256, so one changed bit fails the check.
- **Authentication:** devices prove key ownership; the admin needs password + one-time email code (single-use, 10 min, 5 tries).
- **Key management:** keys come from the OS secure random generator; the master key is split (any 3 of 5 rebuild it, 2 reveal nothing), sealed, and **rotated** if a share leaks; the signing key stays on the server.
- **Threats considered:** spoofing, replay, forged challenge, timing attack, brute force ($2^{256} \approx 1.2\times10^{77}$ keys), stolen key file, leaked share, tampered / attacker-signed / relabelled / downgraded firmware: **all blocked in the demo**.

12. CONCLUSION

- **Key finding:** the server can verify a secret it never receives, using HMAC-SHA256 on a fresh random challenge.
- **Objectives achieved:** spoofed, replayed and forged logins were refused and logged; firmware installs only with a valid signature; **no single person or file holds the master key**, and a leaked share can be cancelled by rotation.
- **Outcome:** a working web app with 88 passing automated tests, including crash-safety of key rotation; the cryptography adds about a microsecond per login.

13. FUTURE SCOPE

- TLS and **mutual authentication** (the device also checks the server)
- Secure element / **HSM** key storage and a secure-boot chain
- **Device key rotation and revocation** (new keys for the devices themselves)
- **Threshold signing**: split the firmware signing key too, plus two-person release approval
- Behaviour-based **anomaly detection**; tamper-evident, hash-chained logs

14. REFERENCES

- W. Stallings, *Cryptography and Network Security*
- RFC 2104: HMAC: Keyed-Hashing for Message Authentication
- RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)
- A. Shamir, *How to Share a Secret*, Communications of the ACM 22(11), 1979
- NIST FIPS 198-1 (HMAC), FIPS 180-4 (SHA-2) and SP 800-57 (key management)